

Literature Review: The Impact of High-Cardinality Data in IoT Time-Series Databases

Existing studies establish that high cardinality is a primary and severe challenge for Time-Series Databases (TSDBs) in Internet of Things (IoT) applications [performance-implications-of-high-cardinality-in-time-series-database]. In this context, cardinality refers to the total number of unique time series being monitored. A unique time series is not just a single metric (like temperature), but the unique combination of that metric and its descriptive key-value metadata, known as tags or labels (e.g., temperature{device_id='xyz-001', location='floor-2', building='A'}). The problem arises directly from the nature of IoT, where the goal is to monitor millions of individual devices, sensors, or endpoints. As the number of devices grows and the metadata used to describe them becomes richer, the number of unique time series explodes, creating the high-cardinality condition [time-series-database-solution-to-the-problem-of-timeline-expansion-high-cardinality].

The most critical impact of high cardinality, as documented across multiple studies, is on the database's indexing system and memory consumption [performance-implications-of-high-cardinality-in-time-series-database]. TSDBs rely on an in-memory index to quickly locate the data for any given time series. As the number of unique series grows into the millions, this index becomes enormous, a condition often called "index bloat." This massive index requires a prohibitive amount of RAM to operate efficiently [performance-implications-of-high-cardinality-in-time-series-database, time-series-database-solution-to-the-problem-of-timeline-expansion-high-cardinality]. When the index can no longer fit in memory, the database is forced to constantly read from the much slower disk, causing performance to plummet and potentially leading to system crashes from Out-Of-Memory (OOM) errors. Research shows that the storage and management of these series keys and their indexes can consume up to 75% of total storage costs in large-scale deployments.

High cardinality also creates significant bottlenecks for both data ingestion (writes) and querying (reads). Write performance degrades because for every incoming data point, the database must perform a lookup in its massive index to see if a new series needs to be created, an operation that becomes increasingly slow and resource-intensive [performance-implications-of-high-cardinality-in-time-series-database]. At extreme scales, the system may fail to keep up with the rate of incoming data, leading to a "death spiral" in performance [performance-implications-of-high-cardinality-in-time-series-database]. Similarly, query latency increases dramatically. Queries that filter data based on tags—a common pattern in IoT analytics—require the database to sift through millions of index entries to find the matching series, a process that becomes exponentially slower as cardinality rises [performance-implications-of-high-cardinality-in-time-series-database].

In response to these challenges, different TSDBs have adopted distinct architectural strategies. InfluxDB evolved from an in-memory index to a disk-based Time Series Index (TSI) that uses an inverted index, a design better suited for managing a large number of series without consuming excessive RAM [performance-implications-of-high-cardinality-in-time-

series-database, time-series-database-solution-to-the-problem-of-timeline-expansion-high-cardinality]. TimescaleDB, an extension of PostgreSQL, leverages the database's mature B-Tree indexing on tag columns and partitions data into manageable chunks to limit the scope of index scans [performance-implications-of-high-cardinality-in-time-series-database]. Newer systems like VictoriaMetrics were designed specifically for high cardinality, employing highly optimized data structures and indexing to provide better performance and lower resource usage compared to older systems in benchmark tests [high-cardinality-tsdb-benchmarks-victoriametrics-vs-timescaledb-vs-influxdb].

Beyond the choice of database, the literature emphasizes that effective data modeling is a critical strategy for mitigating high-cardinality issues at the source [performance-implications-of-high-cardinality-in-time-series-database]. Instead of relying solely on the database to handle an ever-growing number of unique series, best practices involve designing the data schema to control cardinality. Documented techniques include creating hierarchical tag structures to reduce the number of top-level tags, normalizing high-cardinality fields (e.g., storing a subnet instead of a full IP address), and using pre-computed aggregations or materialized views for frequently accessed query patterns [performance-implications-of-high-cardinality-in-time-series-database]. This approach treats cardinality as a design consideration rather than just a technical problem, ensuring that the data model itself contributes to the long-term stability and performance of the IoT data platform.